



Simple Banking DApp - Full Project Report & Guide

1. Project Overview

The Simple Banking DApp is a decentralized Web3 application running on the Ethereum Sepolia Testnet. It allows users to register an account (receiving a bonus of 100 internal Bank Tokens), deposit fake Ethereum (ETH), withdraw ETH, and transfer Bank Tokens to other registered users.

2. Phase 1: Wallet & Environment Setup

2.1 Installing and Configuring MetaMask

1. **Download MetaMask:** Installed the MetaMask browser extension.
2. **Resolve Conflicts:** (Brave Browser specific) Navigated to <brave://settings/wallet> and changed the Default Web3 Wallet to "Extensions" to prevent Brave Wallet from blocking MetaMask.
3. **Network Setup (Chainlist):**
 - MetaMask defaults to the Ethereum Mainnet (real money).
 - Visited chainlist.org, searched for "Sepolia", and clicked "Add to MetaMask". This is a one-time setup that adds the Sepolia Testnet to the wallet's network list.

2.2 Getting Test Funds (Faucet)

1. Visited the **Google Web3 Faucet** (<https://cloud.google.com/application/web3/faucet/ethereum/sepolia>).
 2. Pasted the MetaMask wallet address (Account 1).
 3. Received 0.05 free "Sepolia ETH" to pay for network gas fees during testing.
-

3. Phase 2: Smart Contract Development

3.1 Writing the Code

1. Opened the official Ethereum IDE: **remix.ethereum.org**.
2. Created a new Solidity file named [SimpleBank.sol](#).
3. Wrote the smart contract logic with the following core functions:
 - [registernewaccount\(\)](#): Grants 100 Bank Tokens to new users.
 - [deposit\(\)](#): A payable function allowing users to lock ETH in the contract.
 - [withdraw\(\)](#): Allows users to pull their deposited ETH back into their wallet.
 - [transfer\(\)](#): Allows sending Bank Tokens between two registered addresses.

3.2 Deploying to the Blockchain

1. In Remix, compiled the [SimpleBank.sol](#) code.
2. Navigated to the "Deploy" tab and changed the **Environment** to **"Injected Provider - MetaMask"**.
3. Clicked **Deploy** and approved the gas fee popup in MetaMask.
4. The blockchain successfully created the contract. **Copied the permanent Contract Address** for the next step.

4. Phase 3: Frontend Web Development

4.1 Creating the Interface

1. Created `index.html` in VS Code to act as the user interface.
2. Styled the application using standard CSS.
3. Imported **ethers.js** via CDN. Ethers.js is the crucial library that acts as a bridge between the HTML buttons and the MetaMask extension.

4.2 Linking the Contract

1. Inside the JavaScript section of `index.html`, pasted the newly deployed **Contract Address**.
2. Added the **ABI** (Application Binary Interface) array. This acts as a blueprint, telling the JavaScript exactly what functions exist inside the Solidity contract.

4.3 Auto-Network Switcher (Glitch Fix)

To prevent users from accidentally connecting with the wrong network, we added the following code inside the `connectWallet` function. This automatically forces MetaMask to switch to Sepolia:

```
await window.ethereum.request({
  method: 'wallet_switchEthereumChain',
  params: [{ chainId: '0xaa36a7' }], // 0xaa36a7 is hex for Sepolia
});
```

5. Phase 4: Final Testing & Verification

5.1 Starting the Application

1. Opened VS Code and used the **Live Server** extension.
2. The application securely loaded in the browser at `http://127.0.0.1:5500`.

5.2 Testing Account 1

1. Clicked **Connect MetaMask** and authorized Account 1.
2. Clicked **Register Account** (Received 100 Bank Tokens).
3. Clicked **Deposit ETH** with a value of `0.01` (MetaMask balance went down, Contract balance went up).
4. Clicked **Withdraw ETH** to prove the contract safely returns funds.

5.3 Testing Multi-User Transfers

1. Inside MetaMask, created a brand new address (**Account 2**).
2. Transferred 0.01 Sepolia ETH directly from Account 1 to Account 2 to provide it with gas money.
3. Switched MetaMask to Account 2, refreshed the website, and connected.
4. Clicked **Register Account** so Account 2 received its 100 starting tokens.
5. Copied Account 1's address, pasted it into the **Transfer** box, and sent 10 tokens.

6. **Result:** Account 2 dropped to 90 tokens, and Account 1 successfully increased to 110 tokens!

CONCLUSION: The Web3 DApp architecture is fully functional from end-to-end.